

R5001C CMM i2c MANUAL

Revision: 2.5

Date: September 2023

Prepared	Approved
R&D / Senior Engineer Jonson Jiang	R&D / Supervisor Ares Lu

Revision History

Release	Date	Summary
1.0	2022/11	Start version.
2.0	2023/05	Add fan level and curve control register. Update power off delay default register values. Update CMM firmware version to 9060949752.
2.1	2023/5	Update Table 6-1,6-2 Update Table of contents Update Figure 3-1, 3-3
2.2	2023/7	1. Correct data of Get the Fan Level i2c example. 2. Add Fan Speed Threshold note. 3. Add LED STATE note. 4. SYNC with CMM firmware version 9060949754.
2.3	2023/7	1. Update the Fan Speed Threshold note. 2. Update the chapter 7, Linux i2c command and example
2.4	2023/8	1. Update the Fan Speed Threshold format and note. 2. Update default fan curve set to 8. 3. SYNC with CMM firmware version 9060949756.
2.5	2023/9	1. SYNC with CMM firmware version 9060949757. 2. Add get serial number feature.

Table of Contents

1	<i>Terms and Abbreviations.....</i>	6
2	<i>Introduction.....</i>	7
3	<i>Chassis Management Architecture.....</i>	8
4	<i>Device I2C Address</i>	13
5	<i>Message Format.....</i>	14
5.1	Write Message	14
5.2	Read Message	14
6	<i>CMM Command.....</i>	16
6.1	I2C_CMD_GET_CHASSIS_STATE.....	17
6.2	I2C_CMD_GET_FAN_SPEED.....	18
6.3	I2C_CMD_GET_TEMPERATURE.....	18
6.4	I2C_CMD_GET_VOLTAGE	20
6.5	I2C_CMD_GET_CURRENT.....	21
6.6	I2C_CMD_SET_LED_STATE	21
6.7	I2C_CMD_GET_LED_STATE.....	22
6.8	I2C_CMD_SET_TEMPERATURE_GOING_HIGH_THRESHOLD.....	23
6.9	I2C_CMD_GET_TEMPERATURE_GOING_HIGH_THRESHOLD.....	24
6.10	I2C_CMD_SET_FAN_SPEED_GOING_LOW_THRESHOLD.....	26
6.11	I2C_CMD_GET_FAN_SPEED_GOING_LOW_THRESHOLD.....	26
6.12	I2C_CMD_SET_POWER_OFF_DELAY_TIMER.....	27
6.13	I2C_CMD_GET_POWER_OFF_DELAY_TIMER.....	27
6.14	I2C_CMD_SET_BATTERY_VOLTAGE_STABLE_STAGE_TIMER.....	28
6.15	I2C_CMD_GET_BATTERY_VOLTAGE_STABLE_STAGE_TIMER.....	28
6.16	I2C_CMD_SET_PWR_ON_QUARANTINE_STAGE_TIMER.....	28
6.17	I2C_CMD_GET_PWR_ON_QUARANTINE_STAGE_TIMER.....	29
6.18	I2C_CMD_FACTORY_RESTORE.....	29
6.19	I2C_CMD_GET_MANUFACTURER.....	29
6.20	I2C_CMD_GET_FIRMWARE_VERSION.....	30

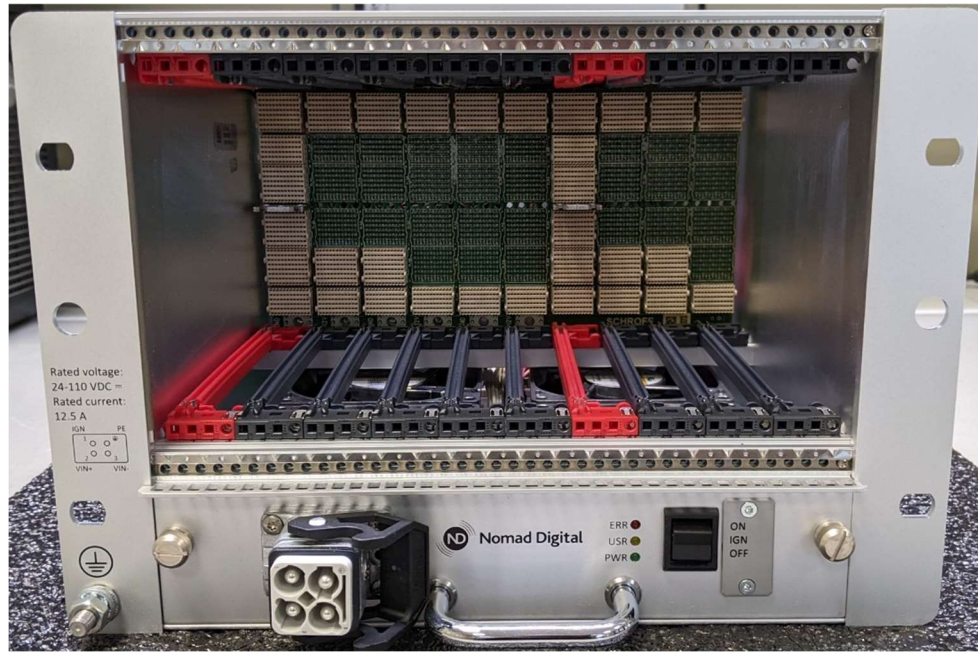
6.21	I2C_CMD_SET_FAN_LEVEL.....	31
6.22	I2C_CMD_GET_FAN_LEVEL.....	32
6.23	I2C_CMD_SET_DEFAULT_FAN_CURVE.....	32
6.24	I2C_CMD_GET_DEFAULT_FAN_CURVE.....	33
6.25	I2C_CMD_GET_SERIAL_NUMBER.....	33
7	Linux i2ctransfer example.....	35
7.1	Linux mount i2c module and detect devices	35
7.2	CMM i2c command code example.....	36
8	Appendix – References	41
9	External References.....	41
TABLE 4-1	ADDRESS OF THE CMM	13
TABLE 6-1	CMM COMMAND LIST.....	16
TABLE 6-2	I2C_CMD_GET_CHASSIS_STATE	17
TABLE 6-3	I2C_CMD_SET_LED_STATE	22
TABLE 6-4	I2C_CMD_GET_LED_STATE	23
TABLE 7-1	CMM I2C COMMAND CODE EXAMPLE	36
FIGURE 3-1	CHASSIS MANAGEMENT ARCHITECTURE	8
FIGURE 3-2	POWER IGNITION TIMING SEQUENCE	10
FIGURE 3-3	POWER IGNITION CONTROL	11
FIGURE 3-4	SOFTWARE FLOW CHART	12
FIGURE 5-1	WRITE MESSAGE INITIATED BY THE MASTER DEVICE	14
FIGURE 5-2	READ MESSAGE INITIATED BY THE MASTER DEVICE WITH STOP AND START CONDITION	14
FIGURE 6-1	I2C_CMD_GET_CHASSIS_STATE.....	17
FIGURE 6-2	I2C_CMD_GET_FAN_SPEED.....	18
FIGURE 6-3	FT_I2C_CMD_GET_TEMPERATURE	18
FIGURE 6-4	I2C_CMD_GET_VOLTAGE.....	20
FIGURE 6-5	I2C_CMD_GET_CURRENT.....	21
FIGURE 6-6	I2C_CMD_SET_LED_STATE.....	21
FIGURE 6-7	I2C_CMD_GET_LED_STATE.....	22
FIGURE 6-8	I2C_CMD_SET_TEMPERATURE_THRESHOLD	23
FIGURE 6-9	I2C_CMD_GET_TEMPERATURE_THRESHOLD.....	24
FIGURE 6-10	I2C_CMD_SET_FAN_SPEED_GOING_LOW_THRESHOLD	26
FIGURE 6-11	I2C_CMD_GET_FAN_SPEED_GOING_LOW_THRESHOLD	26
FIGURE 6-12	I2C_CMD_SET_POWER_OFF_DELAY_TIMER	27
FIGURE 6-13	I2C_CMD_GET_POWER_OFF_DELAY_TIMER	27

FIGURE 6-14 I2C_CMD_SET_BATTERY_VOLTAGE_STABLE_STAGE_TIMER	28
FIGURE 6-15 I2C_CMD_GET_BATTERY_VOLTAGE_STABLE_STAGE_TIMER	28
FIGURE 6-16 I2C_CMD_SET_PWR_ON_QUARANTINE_STAGE_TIMER.....	28
FIGURE 6-17 I2C_CMD_GET_PWR_ON_QUARANTINE_STAGE_TIMER	29
FIGURE 6-18 I2C_CMD_FACTORY_RESTORE.....	29
FIGURE 6-19 I2C_CMD_GET_MANUFACTURER.....	29
FIGURE 6-20 I2C_CMD_GET_FIRMWARE_VERSION.....	30
FIGURE 6-21 I2C_CMD_SET_FAN_LEVEL	31
FIGURE 6-22 I2C_CMD_GET_FAN_LEVEL	32
FIGURE 6-23 I2C_CMD_SET_DEFAULT_FAN_CURVE.....	32
FIGURE 6-24 I2C_CMD_GET_DEFAULT_FAN_CURVE.....	33
FIGURE 6-20 I2C_CMD_GET_SERIAL_NUMBER.....	33

1 Terms and Abbreviations

2 Introduction

This Manual is a technical specification of CMM's i2c register and features, like fan speed control, LED control, voltage and current sensing, ignition signal detect...etc.



3 Chassis Management Architecture

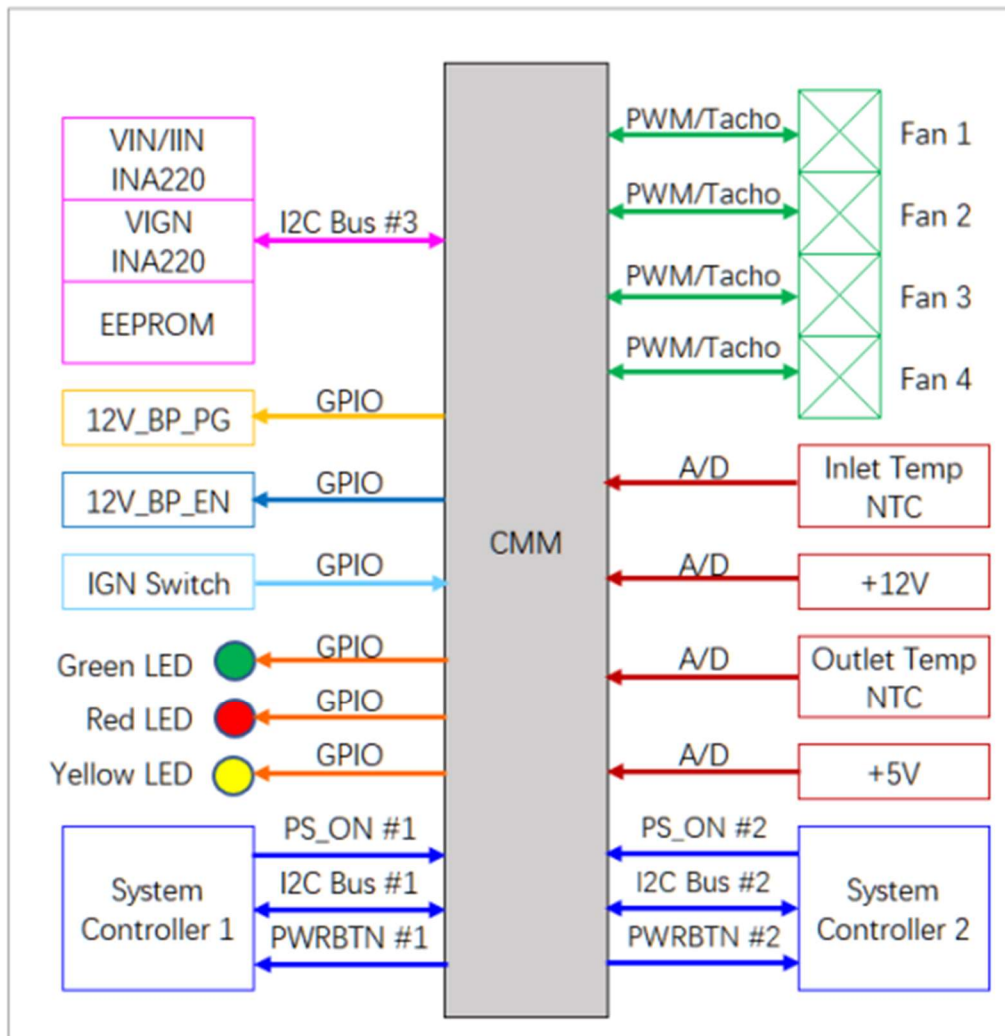


Figure 3-1 Chassis management architecture

The R5001C chassis manager has two dedicated I2C Slave interfaces which connected to the two-system slot separately. The boards in the two system slots could query the chassis state by polling the CMM simultaneously, but they can NOT set the parameters simultaneously, only the board in the Master System slot selected by the System_SEG1_SEG2 slide switch on the chassis front panel could set the parameters to the CMM, CMM will ignore the setting parameter commands from the slave system slot.

The boards in the two system slots could confirm whether itself is the master one by querying the CMM through the I2C bus.

The chassis has four fans which working in full speed mode constantly, the CMM will monitor the fans' speed, if a measured fan speed is less than the fan speed going low threshold (default is 3300 RPM for the standard equipment fan), this fan will be marked as malfunction.

The chassis by default populate two thermometers, one is located at the inlet, and the other is located at the outlet, the boards in the system slots could query the inlet and outlet temperature through the I2C bus.

The chassis monitors the 12V and 5V Standby, according to the PICMG CPCI-S.0, the range of the 12V shall be 11.4V ~ 12.6V, and the range of the 5V standby shall be 4.75V ~ 5.25V, if the measured value of the 12V and the 5V standby is out of the range, this will be marked as a power error.

The chassis support two 12V to backplane power on/off mode, one is the Ignition on/off mode, the other is the manual on/off mode, the boards in the two system slots could query the 12V to backplane state through the I2C bus.

There is a three-position switch on the chassis front panel, when the switch is at the IGN position, the chassis is in Ignition on/off mode, otherwise, it is in the manual on/off mode, when the switch is at ON position, the chassis is power on manually, when the switch is at OFF position, the chassis is power off, the 5V standby output is also power off, so the CMM can NOT work.

In Ignition mode, the CMM measures the VIN and VIGN input, and power on/off the chassis, according to the following power ignition control flow chart, the timing sequence requirement, and the software flow chart.

The shutdown signalling must be provided to both System slots (Slot #1 and slot #7) simultaneously. The preferred signalling to the host OS would be ACPI or a similar approach through USB. Important is that the CPU blades doesn't need special circuits to support Ignition control. The power down timer can be configured into the power supply through SMB/I2C or through BIOS settings on one or both CPU blades. If there is a standard approach for Ignition control in CompactPCI® Serial systems, this must be implemented.

The Ignition control functionality must be designed as follows:

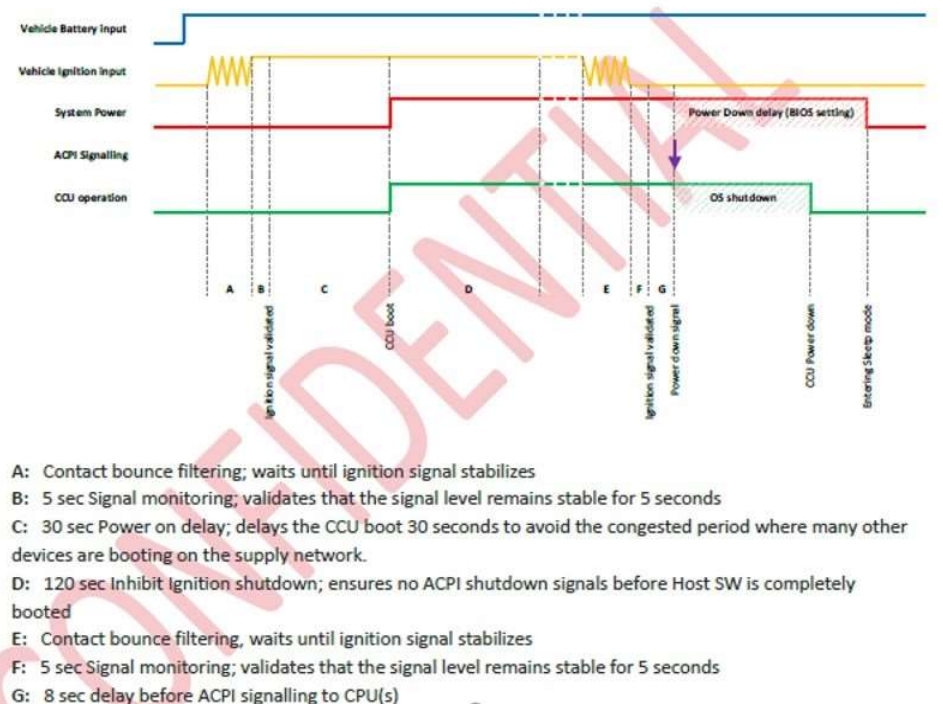


Figure 3-2 Power Ignition Timing Sequence

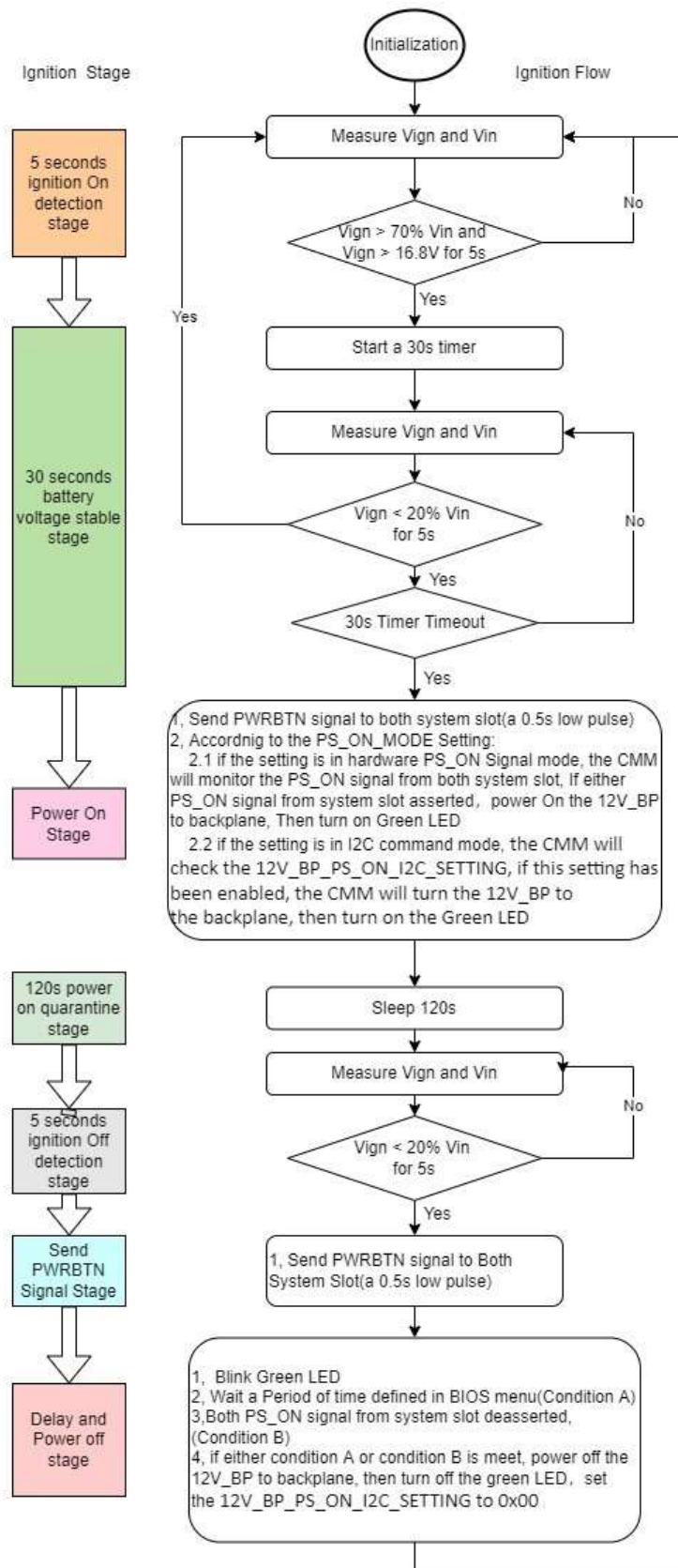


Figure 3-3 Power Ignition Control

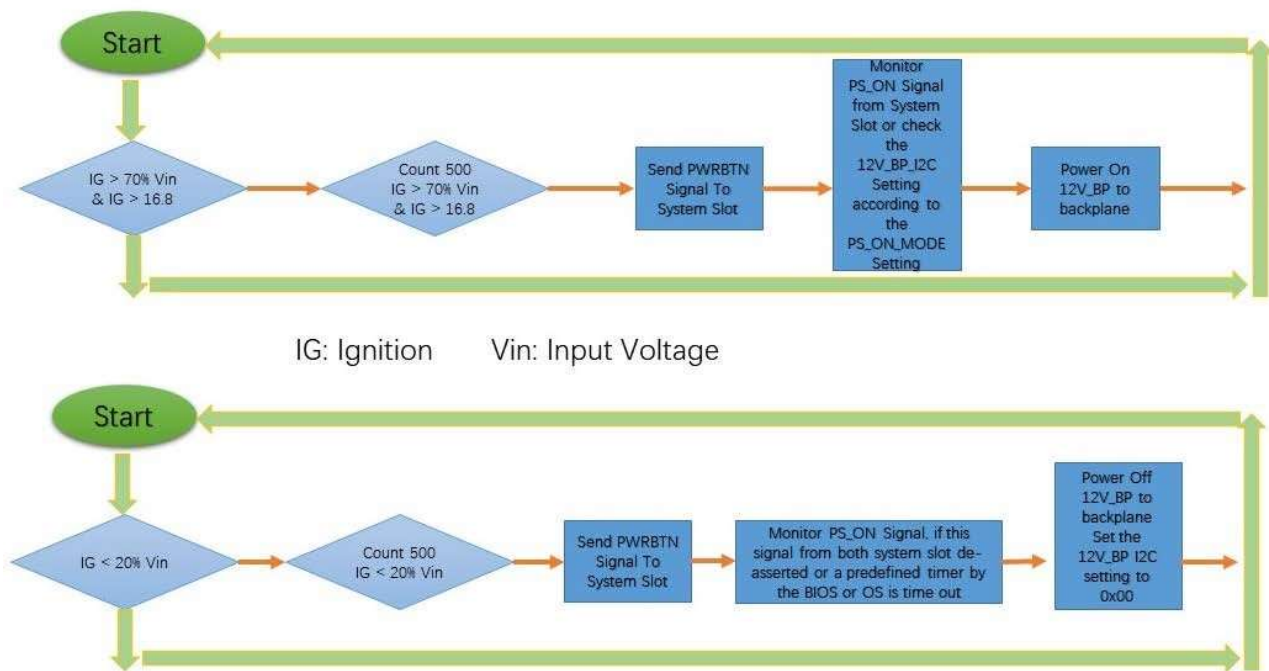


Figure 3-4 Software Flow Chart

There are three LED on the chassis front panel.

The Green LED indicates the 12V_BP Power state, when the 12V_BP to the backplane is enabled, this LED is turned on, when the 12V_BP to the backplane is disabled, this LED is turned off, if the chassis is in a delay and power off stage, this LED is blinking.

The red LED indicates the chassis error state, this led could be controlled locally according to the monitored state by the CMM, also it could be controlled by the board in the master system slot remotely through the I2C Bus. When the CMM detects some abnormal state like 12V or 5V Standby is out of range, or the fans speed is under than the fan speed going low threshold, CMM will turn on this LED, the board in the Master system slot could turn on/off this LED at any time, and the board in the master system slot has the higher privilege to remotely control this LED than the CMM locally.

The Yellow LED is a customized LED, the state of this LED is defined by the customer, the customer can turn on/off this LED through I2C bus by the board in the master system slot when they need, this LED do not have a local state.

4 Device I2C Address

The two dedicated I2C interface which are connected to both system slot separately are designed as Slave only devices, they can only work as slave receiver and slave transmitter. The maximum speed they can support is 100K bps. The two I2C controller use same I2C address.

Table 4-1 *Address of the CMM*

Device Name	Device I2C Address
CMM (7bits/8bits)	0x2D / 0x5A

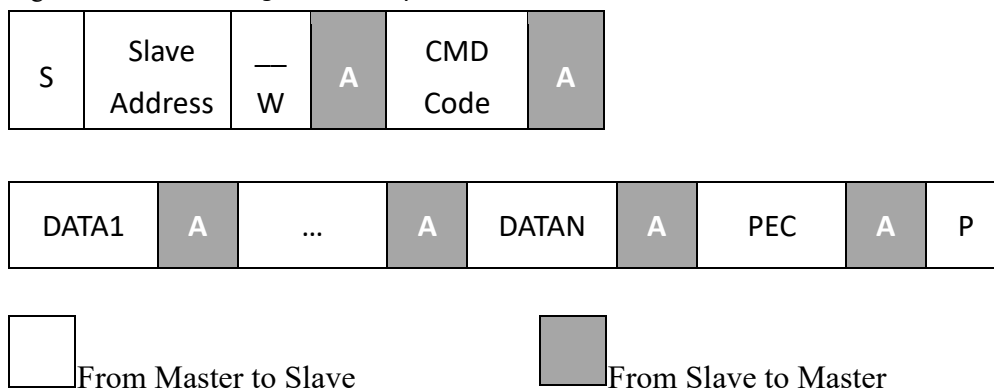
5 Message Format

The CMM Controllers are connected to the backplane through the I2C1 and I2C2 bus channel, both boards located on the two system slots could query the chassis state by polling the CMM, but please be note.

The messages format to control or get information from the CMM are like the following:

5.1 Write Message

Figure 5-1 Write Message initiated by the Master device



PEC, Packet Error Code, the method to calculate the code comply with the I2C bus Specification V2.0, the PEC may be calculated in any way that conforms to a CRC-8 represented by the polynomial,

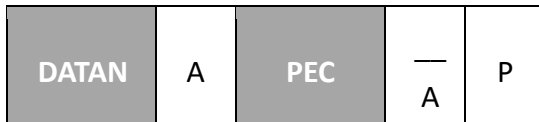
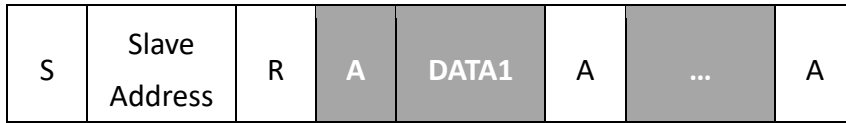
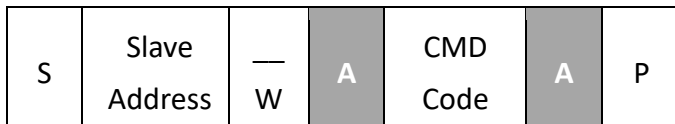
$C_{(X)} = X^8 + X^2 + X^1 + 1$ and must be calculated in the order of the bits are transmitted and include the Slave Address.

Data byte is optional.

5.2 Read Message

If the Master device wants to fetch some information from the slave device, it must firstly initiate a write message to the slave, to tell the slave which information it wants to fetch, then the master should initiate a Read Message to read back these information. After it finished the write message, it can stop this message then initiate a new Start condition to begin the read message, or it can initiate a repeat start condition without stop the write message.

Figure 5-2 Read Message Initiated by the Master device with STOP and START condition



PEC, Packet Error Code, the method to calculate the code comply with the I2C bus Specification V2.0, the PEC may be calculated in any way that conforms to a CRC-8 represented by the polynomial,

$C(x) = X^8 + X^2 + X^1 + 1$ and must be calculated in the order of the bits are received and include the Slave Address after the Repeat Start condition.

6 CMM Command

CMM supports the following commands:

Table 6-1 CMM Command List

Command	Code	Function
I2C_CMD_GET_CHASSIS_STATE	0x01	Get the chassis state
I2C_CMD_GET_FAN_SPEED	0x02	Get the fan speed
I2C_CMD_GET_TEMPERATURE	0x03	Get the temperature
I2C_CMD_GET_VOLTAGE	0x04	Get the voltage
I2C_CMD_GET_CURRENT	0x05	Get the current
I2C_CMD_SET_LED_STATE	0x06	Set LED state
I2C_CMD_GET_LED_STATE	0x07	Get LED state
I2C_CMD_SET_TEMPERATURE_GOING_HIGH_THRESHOLD	0x08	Set temperature going high threshold
I2C_CMD_GET_TEMPERATURE_GOING_HIGH_THRESHOLD	0x09	Get Temperature going high threshold
I2C_CMD_SET_FAN_SPEED_GOING_LOW_THRESHOLD	0x0A	Set fan speed going low threshold
I2C_CMD_GET_FAN_SPEED_GOING_LOW_THRESHOLD	0x0B	Get fan speed going low threshold
I2C_CMD_SET_POWER_OFF_DELAY_TIMER	0x0C	Set the power off delay timer
I2C_CMD_GET_POWER_OFF_DELAY_TIMER	0x0D	Get the power off delay timer
I2C_CMD_SET_BATTERY_VOLTAGE_STABLE_STAGE_TIMER	0x0E	Set the battery voltage stable stage timer
I2C_CMD_GET_BATTERY_VOLTAGE_STABLE_STAGE_TIMER	0x0F	Get the battery voltage stable stage timer
I2C_CMD_SET_PWR_ON_QUARANTINE_STAGE_TIMER	0x10	Set the power on quarantine stage timer
I2C_CMD_GET_PWR_ON_QUARANTINE_STAGE_TIMER	0x11	Get the power one quarantine stage timer
I2C_CMD_FACTORY_RESTORE	0x12	Restore to factory default settings
I2C_CMD_GET_MANUFACTURER	0x13	Query the manufacturer
I2C_CMD_GET_FIRMWARE_VERSION	0x14	Query the firmware version
I2C_CMD_SET_FAN_LEVEL	0x15	Set the fan level(duty cycle)
I2C_CMD_GET_FAN_LEVEL	0x16	Get the fan level(duty cycle)
I2C_CMD_SET_DEFAULT_FAN_CURVE	0x17	Set the fan curve value
I2C_CMD_GET_DEFAULT_FAN_CURVE	0x18	Get the fan curve value
I2C_CMD_GET_SERIAL_NUMBER	0x20	Get the serial number

6.1 I2C_CMD_GET_CHASSIS_STATE

Figure 6-1 I2C_CMD_GET_CHASSIS_STATE

S	Slave Address	— W	A	CMD 0x01	A
---	---------------	--------	---	-------------	---

Sr	Slave Address	R	A	Status 1	A	Status 2	A	PEC	— A	P
----	---------------	---	---	----------	---	----------	---	-----	--------	---

Table 6-2 I2C_CMD_GET_CHASSIS_STATE

Status 1	Bit 7	Fan 1 State	0	OK
			1	Fan speed is lower than the fan speed going low threshold
	Bit 6	Fan 2 State	0	OK
			1	Fan speed is lower than the fan speed going low threshold
	Bit 5	Fan 3 State	0	OK
			1	Fan speed is lower than the fan speed going low threshold
	Bit 4	Fan 4 State	0	OK
			1	Fan speed is lower than the fan speed going low threshold
	Bit 3	12V State	0	OK
			1	12V is not in the range 11.4V- 12.6V
Status 2	Bit 2	5VSTB	0	OK
			1	5V standby is not in the range 4.75V - 5.25V
	Bit 1	Inlet Temperature State	0	OK
			1	Inlet Temperature is higher than the inlet temperature going high threshold
	Bit 0	Outlet Temperature State	0	OK
			1	Outlet Temperature is higher than the outlet temperature going high threshold
	Bit7 - Bit6	12V to Backplane State	00	12V to backplane is OFF, In Ignition Mode(the 3-position switch is at IGN position)
			01	12V to backplane is ON, In Ignition Mode(the 3-position switch is at IGN position)
			10	12V to backplane is ON, In Manual Mode(the 3-position switch is at ON position)
			11	Reserved
Status 2	Bit 5	Reserved	0	Reserved
	Bit 4	Reserved	0	Reserved
	Bit 3	PS_ON 1 State	0	PS_ON State from System slot 1 is LOW
			1	PS_ON State from System slot 1 is HIGH
	Bit 2	PS_ON 2 State	0	PS_ON State from System slot 2 is LOW
			1	PS_ON State from System slot 2 is HIGH
	Bit 1	12V_BP Power Good Signal	0	12V to backplane is good
			1	12V to backplane is not good
Status 2	Bit 0	Reserved	0	Reserved

6.2 I2C_CMD_GET_FAN_SPEED

Figure 6-2 I2C_CMD_GET_FAN_SPEED

S	Slave Address	— W	A	CMD 0x02	A
---	---------------	--------	---	-------------	---

Sr	Slave Address	R	A	M=0x3C	A	Fan 1 Speed	A
----	---------------	---	---	--------	---	-------------	---

Fan 2 Speed	A	Fan 3 Speed	A	Fan 4 Speed	A	PEC	— A	P
-------------	---	-------------	---	-------------	---	-----	--------	---

M 2's complement, signed, it always equal 0x3C (60 in decimal)

The “raw” fan speed can be converted to the real fan speed in “RPM” units through the following formula:

Fan 1 Real speed (RPM) = Fan 1 Speed * M

Fan 2 Real speed (RPM) = Fan 2 Speed * M

Fan 3 Real speed (RPM) = Fan 3 Speed * M

Fan 4 Real speed (RPM) = Fan 4 Speed * M

6.3 I2C_CMD_GET_TEMPERATURE

Figure 6-3 FT_I2C_CMD_GET_TEMPERATURE

S	Slave Address	— W	A	CMD 0x03	A
---	---------------	--------	---	-------------	---

Sr	Slave Address	R	A	Inlet Temperature MSB	A	Inlet Temperature LSB	A
----	---------------	---	---	--------------------------	---	--------------------------	---

Outlet Temperature MSB	A	Outlet Temperature LSB	A	Reserved for Heatsink MSB	A
---------------------------	---	------------------------------	---	------------------------------	---

Reserved for Heatsink LSB	A	PEC	— A	P
------------------------------	---	-----	--------	---

The Inlet Temperature | Outlet Temperature are 16 bits signed short type value in 2's complement format, the most significant bytes are transferred first, they can be transferred to real temperature in degree unit through the following formula:

Inlet Temperature in degree =

((short)((Inlet Temperature MSB << 8 | Inlet Temperature LSB)) / 10.0F)

Outlet Temperature in degree = ((short) ((Outlet Temperature MSB << 8 | Outlet Temperature LSB)) / 10.0F)

For example, Inlet Temperature MSB= 0xFF, Inlet Temperature LSB= 0x9B, Outlet Temperature MSB = 0x01, Outlet Temperature LSB = 0x90, Then,

Inlet Temperature (in Degree) = ((short) ((0xFF << 8 | 0x9B)) / 10.0F = ((short)0xFF9B) / 10.0F = (-101) / 10.0F = -10.1

Outlet Temperature (in Degree) = ((short) ((0x01 << 8 | 0x90)) / 10.0F = ((short)0x0190) / 10.0F = 400 / 10.0F = 40.0

The “Reserved for Heatsink MSB” and the “Reserved for Heatsink LSB” are always 0xFF, it has no sense in current version, these two bytes are reserved for future possible use cases.

6.4 I2C_CMD_GET_VOLTAGE

Figure 6-4 I2C_CMD_GET_VOLTAGE

S	Slave Address	— W	A	CMD 0x04	A
---	---------------	--------	---	-------------	---

Sr	Slave Address	R	A	12V MSB	A	12V LSB	A
----	---------------	---	---	------------	---	---------	---

5V STB MSB	A	5V STB LSB	A	Vin MSB	A	Vin LSB	A
---------------	---	---------------	---	---------	---	---------	---

Vign MSB	A	Vign LSB	A	PEC	— A	P
----------	---	----------	---	-----	--------	---

The “raw” value of (12V|5V STB|Vin|Vign) can be converted to the real value in “Volts” units through the following formula:

(12V|5V STB|Vin|Vign) Real Voltage (Volts) =

(12V|5V STB|Vin|Vign) MSB * 256 + (12V|5V STB|Vin|Vign) LSB

For example, 12V MSB= 0x04(4 in decimal), 12V LSB= 0xAF (175 in decimal) Then, the 12V (in voltage) = (4 * 256 + 175) / 100 = 11.99 (Volts)

Another example, Vign MSB = 0x2A (42 in decimal), Vign LSB = 0x94(148 in decimal) Then, the Vign (in voltage) = (42 * 256 + 148) / 100 = 109(Volts)

6.5 I2C_CMD_GET_CURRENT

Figure 6-5 I2C_CMD_GET_CURRENT

S	Slave Address	— W	A	CMD 0x05	A
---	---------------	--------	---	-------------	---

Sr	Slave Address	R	A	Current MSB	A	Current LSB	A	PEC	— A	P
----	---------------	---	---	-------------	---	-------------	---	-----	--------	---

The “raw” value of current can be converted to the real value in “Amp” units through the following formula: Real Current (Amp) = (Current MSB * 256 + Current LSB) / 100.0F

For example, Current MSB= 0x07(7 in decimal), 12V LSB= 0x21(33 in decimal) The current (in Amp) = (7 * 256 + 33) / 100.0F = 18.25

6.6 I2C_CMD_SET_LED_STATE

Figure 6-6 I2C_CMD_SET_LED_STATE

S	Slave Address	— W	A	CMD 0x06	A	LED ID	A	LED Function	A
---	---------------	--------	---	-------------	---	--------	---	--------------	---

On Duration	A	PEC	A	P
-------------	---	-----	---	---

Table 6-3 I2C_CMD_SET_LED_STATE

Byte	Data Definition
LED ID	00h = Green LED
	01h = Red LED
	02h = Yellow LED
	03h - FEh = Reserved
	FFh = All LEDs are addressed
LED Function	00h = LED off Override
	01h - FAh = LED BLINKING override. The off duration is specified by the value of this byte and the on duration is specified by the value of byte Off Duration. Both values specify the time in tens of milliseconds (10 ms –2.5 s)
	FBh = Reserved
	FCh = LED state restored to Local Control state
	FDh - FEh = Reserved
	FFh = LED on Override
On Duration	LED on-time in tens of milliseconds if (1 ≤ LED Function Byte ≤ FAh) Otherwise, this field is ignored and is set to 0h.

Note: LED override priority higher than local control.

6.7 I2C_CMD_GET_LED_STATE

Figure 6-7 I2C_CMD_GET_LED_STATE

S	Slave Address	— W	A	CMD 0x07	A	LED ID	A
---	---------------	--------	---	-------------	---	--------	---

Sr	Slave Address	R	A	LED States	A	Local Control LED Function	A
----	---------------	---	---	------------	---	----------------------------	---

Local Control On Duration	A	Override State LED Function	A	Override State On Duration	A	PEC	— A	P
---------------------------	---	-----------------------------	---	----------------------------	---	-----	--------	---

Table 6-4 I2C_CMD_GET_LED_STATE

Byte	Data Definition
LED States	[7:2] - Reserved
	[1] - 1b if override state has been enabled
	[0] - 1b if CMM has a Local Control state
Local Control LED Function	00h = LED is off. This is the default value if the CMM does not support Local Control, like the Yellow LED.
	01h - FAh = LED is BLINKING. The off duration is specified by the value of this bytes, and the on duration is specified by the value of "Local Control On Duration", Both values specify the time in tens of milliseconds(10ms - 2.5s)
	FBh - FEh = Reserved
	FFh = LED is on
Local Control On Duration	The LED on-time in tens of milliseconds if (01h ≤ Local Control LED Function Byte ≤ FAh). This byte is ignored and is set to 0h if the Local Control LED Function byte is not a value in range 01h - FAh
Override state LED Function	00h = LED override state is off.
	01h - FAh = LED override state is BLINKING. The off duration is specified by the value of this byte and the on duration is specified by the value of "Override State On Duration". Both values specify the time in tens of milliseconds (10 ms–2.5 s).
	FBh - FEh = Reserved
	FFh = LED override state is on
Override Control On Duration	The LED on-time in tens of milliseconds if (01h ≤ Override Control LED Function Byte ≤ FAh). This byte is ignored and is set to 0h if the Override Control LED Function byte is not a value in range 01h - FAh

Note: LED override priority higher than local control.

6.8 I2C_CMD_SET_TEMPERATURE_GOING_HIGH_THRESHOLD

Figure 6-8 I2C_CMD_SET_TEMPERATURE_THRESHOLD

S	Slave Address	— W	A	CMD	A
				0x8	

Inlet Temperature MSB	A	Inlet Temperature LSB	A
-----------------------	---	-----------------------	---

Outlet Temperature MSB	A	Outlet Temperature LSB	A
------------------------	---	------------------------	---

Reserved Heatsink Temperature MSB	A	Reserved for Heatsink LSB	A	PEC	A	P
-----------------------------------	---	---------------------------	---	-----	---	---

The values of (“Inlet Temperature” |” Outlet Temperature” | ”Reserved Heatsink Temperature”) are signed short integer values, for example, if the “Inlet Temperature MSB” value is 0xE2, and the then the real temperature is -30 degree C.

The default inlet temperature going high threshold is 70 degree C, the outlet temperature going high threshold is 85-degree C.

Please Note: The “Reserved for heatsink” is reserved, it can be any value, it has no sense if the temperature sensor is not populated.

6.9 I2C_CMD_GET_TEMPERATURE_GOING_HIGH_THRESHOLD

Figure 6-9 I2C_CMD_GET_TEMPERATURE_THRESHOLD

S	Slave Address	— W	A	CMD 0x9	A
---	---------------	--------	---	------------	---

Sr	Slave Address	R	A	Inlet Temperature Threshold MSB	A	Inlet Temperature Threshold LSB	A
----	---------------	---	---	---------------------------------	---	---------------------------------	---

Outlet Temperature Threshold MSB	A	Outlet Temperature Threshold LSB	A
----------------------------------	---	----------------------------------	---

Reserved Heatsink Temperature Threshold MSB	A	Reserved Heatsink Temperature Threshold LSB	A
---	---	---	---

PEC	— A	P
-----	--------	---

The Inlet Temperature Threshold | Outlet Temperature Threshold are 16 bits signed short type value in 2’s complement format, the most significant bytes are transferred first, they can get from real temperature in degree unit through the following formula:

Inlet Temperature MSB =(((short) (Inlet Temperature Threshold * 10)) >> 8) & 0xFF Inlet
Temperature LSB =((short) (Inlet Temperature Threshold * 10)) & 0xFF
Outlet Temperature MSB =(((short) (Outlet Temperature Threshold * 10)) >> 8) &
0xFF Outlet Temperature LSB =((short) (Outlet Temperature Threshold * 10)) &
0xFF

For example, Inlet Temperature Threshold = -10.1 in degree, Outlet Temperature Threshold = 40.0
in degree, Then,

Inlet Temperature MSB =(((short) (-10.1 * 10)) >> 8) & 0xFF = (0xFF9B >> 8) &
0xFF = 0xFF Inlet Temperature LSB =((short) (-10.1 * 10)) & 0xFF = (0xFF9B >> 8)
& 0xFF = 0x9B
Inlet Temperature MSB =(((short) (40.0 * 10)) >> 8) & 0xFF = (0x0190 >> 8) &
0xFF = 0x01 Inlet Temperature LSB =((short) (40.0 * 10)) & 0xFF = (0x0190 >> 8)
& 0xFF = 0x90

The “Reserved for Heatsink MSB” and the “Reserved for Heatsink LSB” can be any value, it has no
sense in current version, these two bytes are reserved for future possible use cases.

6.10 I2C_CMD_SET_FAN_SPEED_GOING_LOW_THRESHOLD

Figure 6-10 I2C_CMD_SET_FAN_SPEED_GOING_LOW_THRESHOLD

S	Slave Address	— W	A	CMD 0xA	A
---	---------------	--------	---	------------	---

M (0x3C)	A	Local Fan Speed Threshold	A	Manual Fan Speed Threshold	A	PEC	A	P
-------------	---	---------------------------	---	----------------------------	---	-----	---	---

M 2's complement, signed, it always equal 0x3C (60 in decimal)

The “raw” value of the “Fan Speed Threshold” can be converted from the real fan speed in “RPM” units through the following formula:

Fan speed threshold in raw value = Fan Speed Threshold (in RPM) / M

For example, if a user wants to set fan speed going low threshold to 3000 RPM, the raw value of the “Fan Speed Threshold” equal to 3000(RPM) / 60, that is 50 in decimal, 0x32 in hexadecimal.

Please Note:

1. All the four fans use the same fan speed going low threshold.
2. When the fan is running in local_curve_control mode, customer can only change the min. threshold under 2000rpm. But not over 2000rpm .the value over 2000rpm can't be saved. When the fan is running in the override_control mode, customer can set fan speed going low threshold, but the fan speed going low threshold set no higher than Current fan speed.

6.11 I2C_CMD_GET_FAN_SPEED_GOING_LOW_THRESHOLD

Figure 6-11 I2C_CMD_GET_FAN_SPEED_GOING_LOW_THRESHOLD

S	Slave Address	— W	A	CMD 0xB	A
---	---------------	--------	---	------------	---

Sr	Slave Address	R	A	M (0x3C)	A	Local Fan Speed Threshold	A	Manual Fan Speed Threshold	A	PEC	— A	P
----	---------------	---	---	-------------	---	---------------------------	---	----------------------------	---	-----	--------	---

M 2's complement, signed, it always equal 0x3C (60 in decimal)

The “raw” fan speed threshold can be converted to the real fan speed in “RPM” units through the following formula:

$$\text{Fan Speed Threshold (in RPM)} = \text{Fan Speed Threshold (raw value)} * M$$

For example, the Fan Speed Threshold raw value is 30 in decimal, the fan speed threshold in RPM equal to 30 * 60, that is 1800(RPM).

Please Note:

1. All the four fans use the same fan speed going low threshold.

6.12 I2C_CMD_SET_POWER_OFF_DELAY_TIMER

Figure 6-12 I2C_CMD_SET_POWER_OFF_DELAY_TIMER

S	Slave Address	— W	A	CMD 0x0C	A	Power Off Delay Timer	A	PEC	A	P
---	---------------	--------	---	-------------	---	--------------------------	---	-----	---	---

The default value of the power off delay timer is 4 in minutes unit.

6.13 I2C_CMD_GET_POWER_OFF_DELAY_TIMER

Figure 6-13 I2C_CMD_GET_POWER_OFF_DELAY_TIMER

S	Slave Address	— W	A	CMD 0xD	A
---	---------------	--------	---	------------	---

Sr	Slave Address	R	A	Power Off Delay Timer	A	PEC	— A	P
----	---------------	---	---	--------------------------	---	-----	--------	---

The power off delay timer byte is a value in minute unit, the default value is 4.

6.14 I2C_CMD_SET_BATTERY_VOLTAGE_STABLE_STAGE_TIMER

Figure 6-14 I2C_CMD_SET_BATTERY_VOLTAGE_STABLE_STAGE_TIMER

S	Slave Address	— W	A	CMD 0x0E	A	Battery Voltage Stable Stage Timer	A	PEC	A	P
---	---------------	--------	---	-------------	---	--	---	-----	---	---

The default value of the battery voltage stable stage timer byte is 30 in seconds unit.

6.15 I2C_CMD_GET_BATTERY_VOLTAGE_STABLE_STAGE_TIMER

Figure 6-15 I2C_CMD_GET_BATTERY_VOLTAGE_STABLE_STAGE_TIMER

S	Slave Address	— W	A	CMD 0x0F	A
---	---------------	--------	---	-------------	---

Sr	Slave Address	R	A	Battery Voltage Stable Stage Timer	A	PEC	— A	P
----	---------------	---	---	---------------------------------------	---	-----	--------	---

The battery voltage stable stage timer byte is a value in seconds unit, the default value is 30.

6.16 I2C_CMD_SET_PWR_ON_QUARANTINE_STAGE_TIMER

Figure 6-16 I2C_CMD_SET_PWR_ON_QUARANTINE_STAGE_TIMER

S	Slave Address	— W	A	CMD 0x10	A	Pwr on Quarantine Stage Timer	A	PEC	A	P
---	---------------	--------	---	-------------	---	-------------------------------------	---	-----	---	---

The default value of the power on quarantine stage timer byte is 120 in seconds unit.

6.17 I2C_CMD_GET_PWR_ON_QUARANTINE_STAGE_TIMER

Figure 6-17 I2C_CMD_GET_PWR_ON_QUARANTINE_STAGE_TIMER

S	Slave Address	— W	A	CMD 0x11	A
---	---------------	--------	---	-------------	---

Sr	Slave Address	R	A	Pwr on Quarantine Stage Timer	A	PEC	— A	P
----	---------------	---	---	-------------------------------	---	-----	--------	---

The power on quarantine stage timer byte is a value in seconds unit, the default value is 120.

6.18 I2C_CMD_FACTORY_RESTORE

Figure 6-18 I2C_CMD_FACTORY_RESTORE

S	Slave Address	— W	A	CMD 0x12	A	PEC	A	P
---	---------------	--------	---	-------------	---	-----	---	---

This command will restore the factory default settings, including the power off delay timer, the default fan speed going low threshold, the default inlet/outlet temperature going high threshold, etc.

6.19 I2C_CMD_GET_MANUFACTURER

Figure 6-19 I2C_CMD_GET_MANUFACTURER

S	Slave Address	— W	A	CMD 0x13	A
---	---------------	--------	---	-------------	---

Sr	Slave Address	R	A	ID1= 'A'(0x41)	A	ID2= 'D'(0x44)	A
----	---------------	---	---	-------------------	---	-------------------	---

ID3= 'L'(0x4C)	A	ID4= 'I'(0x49)	A	ID5= 'N'(0x4E)	A	ID6= 'K'(0x4B)	A
-------------------	---	-------------------	---	-------------------	---	-------------------	---

PEC	— A	P
-----	--------	---

ID1 = 'A', 0x41 in hexadecimal

ID2 = 'D', 0x44 in hexadecimal

ID3 = 'L', 0x4C in hexadecimal

ID4 = 'I', 0x49 in hexadecimal

ID5 = 'N', 0x4E in hexadecimal

ID6 = 'K', 0x4B in hexadecimal

6.20 I2C_CMD_GET_FIRMWARE_VERSION

Figure 6-20 I2C_CMD_GET_FIRMWARE_VERSION

S	Slave Address		— W	A	CMD 0x14	A
---	------------------	--	--------	---	-------------	---

Sr	Slave Address	R	A	ID1= '9'(0x39)	A	ID2= '0'(0x30)	A	ID3= '6'(0x36)	A	ID4= '0'(0x30)	A
----	------------------	---	---	-------------------	---	-------------------	---	-------------------	---	-------------------	---

ID5 '9'(0x39)	A	ID6= '4'(0x34)	A	ID7= '9'(0x39)	A	ID8= '7'(0x37)	A	ID9= '5'(0x35)	A
------------------	---	-------------------	---	-------------------	---	-------------------	---	-------------------	---

ID10= '6'(0x32)	A	PEC	— A	P
--------------------	---	-----	--------	---

ID1 = '9', 0x39 in hexadecimal

ID2 = '0', 0x30 in hexadecimal

ID3 = '6', 0x36 in hexadecimal

ID4 = '0', 0x30 in hexadecimal

ID5 = '9', 0x39 in hexadecimal

ID6 = '4', 0x34 in hexadecimal

ID7 = '9', 0x39 in hexadecimal

ID8 = '7', 0x37 in hexadecimal
 ID9 = '5', 0x35 in hexadecimal
 ID10 = '6', 0x34 in hexadecimal

6.21 I2C_CMD_SET_FAN_LEVEL

Figure 6-21 I2C_CMD_SET_FAN_LEVEL

S	Slave Address	— W	A	CMD 0x15	A	Override Fan Level	A	PEC	A	P
---	---------------	--------	---	-------------	---	--------------------	---	-----	---	---

Byte	Data Definition
Override Fan Level	0xFF = Local Control
	0xFE = Shut Down
	Greater than or equal to 0x1E(30 in decimal) and less than or equal to 0x64(100 in decimal), if the CMM received a command with a fan level little than 0x1E, it will treat this value as 0x1E, if the CMM received a command with a fan level greater than 0x64, and not equal to 0xFF and 0xFE, it will treat this value as 0x64.

6.22 I2C_CMD_GET_FAN_LEVEL

Figure 6-22 I2C_CMD_GET_FAN_LEVEL

S	Slave Address	— W	A	CMD 0x16	A
---	---------------	--------	---	-------------	---

Sr	Slave Address	R	A	Override Fan Level	A	Local Control Fan Level	A	PEC	— A	P
----	---------------	---	---	--------------------	---	-------------------------	---	-----	--------	---

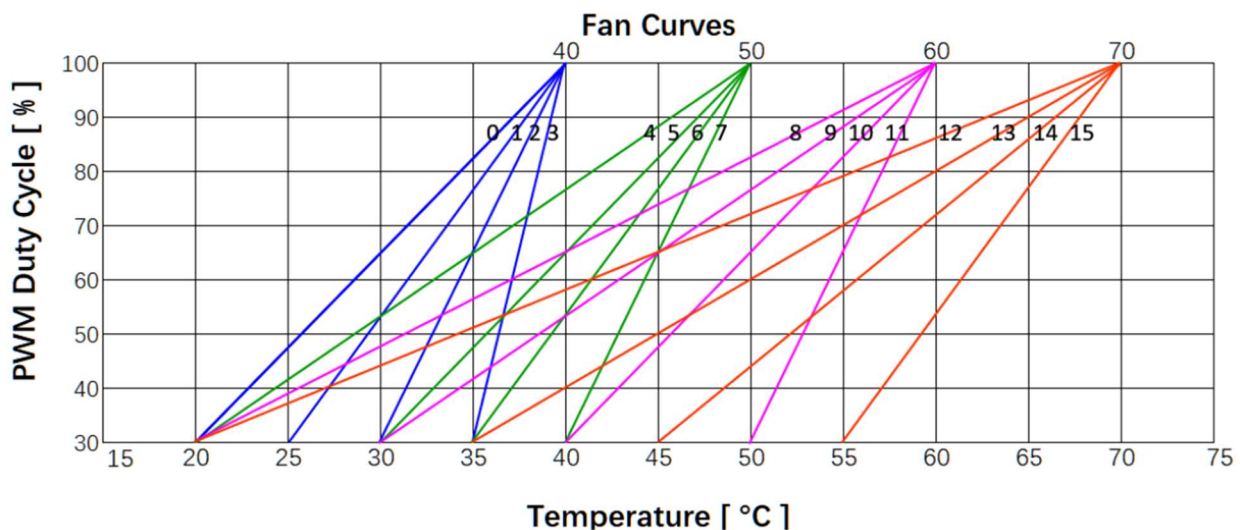
Byte	Data Definition
Override Fan Level	0xFF = Local Control 0xFE = Shut Down Greater than or equal to 0x1E(30 in decimal) and less than or equal to 0x64(100 in decimal)
Local Control Fan Level	If Override Fan Level is 0xFF, this byte is the actual fan level determined by the CMM according to the default fan curves, the value is between 0x1E and 0x64. If the Override Fan Level is 0xFE, the actual fan level is Shut Down. If the Override Fan Level is not equal 0xFE and 0xFF, the actual fan level is the Override Fan Level.

6.23 I2C_CMD_SET_DEFAULT_FAN_CURVE

Figure 6-23 I2C_CMD_SET_DEFAULT_FAN_CURVE

S	Slave Address	— W	A	CMD 0x17	A	Default Fan Curve	A	PEC	A	P
---	---------------	--------	---	-------------	---	-------------------	---	-----	---	---

The default fan curve can be 0 through 15 in decimal, see bellow chart:



6.24 I2C_CMD_GET_DEFAULT_FAN_CURVE

Figure 6-24 I2C_CMD_GET_DEFAULT_FAN_CURVE

d	Slave Address	— W	A	CMD 0x18	A
---	---------------	--------	---	-------------	---

Sr	Slave Address	R	A	Default Fan Curve	A	PEC	— A	P
----	---------------	---	---	-------------------	---	-----	--------	---

By default, fan curve is 8 in decimal

6.25 I2C_CMD_GET_SERIAL_NUMBER

Figure 6-25 I2C_CMD_GET_SERIAL_NUMBER

S	Slave Address	— W	A	CMD 0x14	A
---	---------------	--------	---	-------------	---

Sr	Slave Address	R	A	SN1	A	SN2	A	SN3	A	SN4	A
----	---------------	---	---	-----	---	-----	---	-----	---	-----	---

SN5	A	SN6	A	SN7	A	SN8	A	SN9	A
-----	---	-----	---	-----	---	-----	---	-----	---

SN10	A	SN11	A	SN12	A	PEC	— A	P
------	---	------	---	------	---	-----	--------	---

The user can read the chassis serial number through this command.

For example: the serial number is: 1380600001AA

SN1 = '1', 0x31 in hexadecimal

SN2 = '3', 0x33 in hexadecimal

SN3 = '8', 0x38 in hexadecimal
SN4 = '0', 0x30 in hexadecimal
SN5 = '6', 0x36 in hexadecimal
SN6 = '0', 0x30 in hexadecimal
SN7 = '0', 0x30 in hexadecimal
SN8 = '0', 0x30 in hexadecimal
SN9 = '0', 0x30 in hexadecimal
SN10 = '1', 0x31 in hexadecimal
SN11 = 'A', 0x41 in hexadecimal
SN12 = 'A', 0x41 in hexadecimal

7 Linux i2ctransfer example

7.1 Linux mount i2c module and detect devices

Mount i2c module:

```
$ sudo modprobe i2c-dev
```

List i2c bus:

```
$ sudo i2cdetect -l
```

There can be find out two of 'Synopsys DesignWare I2C adapter' i2c bus name and choose the big one.

For example, we checkout i2c bus 3 device list and discover the CMM address 0x2D in the i2c list table.

```
$ sudo i2cdetect -y -r 3
```

Automatic get the i2c bus number

```
# Probe the i2c-bus driver: i2c-dev
```

```
$ test -z $(lsmod | grep i2c_dev | sed 's/i2c_dev*/i2c_dev/g') && sudo modprobe i2c-dev
```

```
# Get I2C bus system node number
```

```
$ export BUS1=$(sudo i2cdetect -l | grep -i Synopsys | sed 's/i2c-//g' | sed 's/ *i2c *.*$/g' | sed -n "1p")
```

```
$ export BUS2=$(sudo i2cdetect -l | grep -i Synopsys | sed 's/i2c-//g' | sed 's/ *i2c *.*$/g' | sed -n "2p")
```

```
$ test $BUS1 -gt $BUS2 && export I2C=$BUS1
```

```
$ test $BUS2 -gt $BUS1 && export I2C=$BUS2
```

Now we can access CMM via i2c bus \$I2C by i2ctransfer tool

Example:

```
$ sudo i2ctransfer -y $I2C w1@0x2d 0x01 r3
```

PEC CRC-8 CALC URL:

<https://www.1ddgo.net/en/encrypt/crc>

7.2 CMM i2c command code example

Table 7-1 CMM i2c command code example

Command	Code	Example
Get the chassis state	0x01	\$ i2ctransfer -y \$I2C w1@0x2d 0x01 r3 Response: 0x00 0x50 0x7f Convert to bit: 0b00000000, 0b01010000 Status0: 0b00000000(all OK) Status1: 0b01010000(12v to backplane on, ignition mode.....) PEC: 0x7f(0x5b 0x00 0x50)
Get the fan speed	0x02	\$ i2ctransfer -y \$I2C w1@0x2d 0x02 r6 Response: 0x3c 0x64 0x69 0x62 0x6b 0x27 M always are 0x3c (Dec: 60) Fan 1 speed = M * Value= 0x3c * 0x64 = 6000 rpm Fan 2 speed = M * Value = 0x3c * 0x69 = 6300 rpm Fan 3 speed = M * Value = 0x3c * 0x62 = 5880 rpm Fan 4 speed = M * Value = 0x3c * 0x6b = 6420 rpm PEC: 0x27(0x5b 0x3c 0x64 0x69 0x62 0x6b)
Get the temperature	0x03	\$ i2ctransfer -y \$I2C w1@0x2d 0x03 r7 Response: 0x01 0x18 0x01 0x27 0xff 0xff 0x7f Inlet temperature: 0x0118/10 (Dec Signed 2's complement: 280/10 = 28°C) Outlet temperature: 0x0126/10 (Dec Signed 2's complement: 294/10 = 29.4°C) Reserved Heatsink: 0xffff(N/A) PEC: 0x7f(0x5b 0x01 0x18 0x01 0x27 0xff 0xff) Note: HEX convert to DEC Signed 2's calc URL: https://www.rapidtables.com/convert/number/hex-to-decimal.html
Get the voltage	0x04	\$ i2ctransfer -y \$I2C w1@0x2d 0x04 r9 Response: 0x04 0xa4 0x01 0xee 0x09 0x4d 0x06 0xa0 0x21 12V Voltage = Value / 100 = 0x04a4 / 100 = 11.88V

		5V STB Voltage = Value / 100 = 0x01ee / 100 = 4.94V Vin Voltage = Value / 100 = 0x094d / 100 = 23.81V Vign Voltage = Value / 100 = 0x06a0 / 100 = 16.69V PEC: 0x21(0x5b 0x04 0xa4 0x01 0xee 0x09 0x4d 0x06 0xa0)																	
Get the current	0x05	\$ i2ctransfer -y \$I2C w1@0x2d 0x05 r3 Response: 0x01 0x13 0xa4 Current = Value / 100 = 0x0113 / 100 = 2.75A PEC: 0xa4(0x5b 0x01 0x13)																	
Set LED state	0x06	\$ i2ctransfer -y \$I2C w5@0x2d 0x06 0xff 0x5a 0x0a 0xc0 LED ID = 0xff (ALL) LED FUNCTION = 0x5a (DEC 90ms) On Duration = 0x0a (DEC 10ms) PEC: 0xc0 (0x5a 0x06 0xff 0x5a 0x0a) <table><tr><th>Byte</th><th>Data Definition</th></tr><tr><td rowspan="5">LED ID</td><td>00h = Green LED</td></tr><tr><td>01h = Red LED</td></tr><tr><td>02h = Yellow LED</td></tr><tr><td>03h - FEh = Reserved</td></tr><tr><td>FFh = All LEDs are addressed</td></tr><tr><td rowspan="6">LED Function</td><td>00h = LED off Override</td></tr><tr><td>01h - FAh = LED BLINKING override. The off duration is specified by the value of this byte and the on duration is specified by the value of byte Off Duration. Both values specify the time in tens of milliseconds (10 ms –2.5 s)</td></tr><tr><td>FBh = Reserved</td></tr><tr><td>FCh = LED state restored to Local Control state</td></tr><tr><td>FDh - FEh = Reserved</td></tr><tr><td>FFh = LED on Override</td></tr><tr><td>On Duration</td><td>LED on-time in tens of milliseconds if (1 ≤ LED Function Byte ≤ FAh) Otherwise, this field is ignored and is set to 0h.</td></tr></table>	Byte	Data Definition	LED ID	00h = Green LED	01h = Red LED	02h = Yellow LED	03h - FEh = Reserved	FFh = All LEDs are addressed	LED Function	00h = LED off Override	01h - FAh = LED BLINKING override. The off duration is specified by the value of this byte and the on duration is specified by the value of byte Off Duration. Both values specify the time in tens of milliseconds (10 ms –2.5 s)	FBh = Reserved	FCh = LED state restored to Local Control state	FDh - FEh = Reserved	FFh = LED on Override	On Duration	LED on-time in tens of milliseconds if (1 ≤ LED Function Byte ≤ FAh) Otherwise, this field is ignored and is set to 0h.
Byte	Data Definition																		
LED ID	00h = Green LED																		
	01h = Red LED																		
	02h = Yellow LED																		
	03h - FEh = Reserved																		
	FFh = All LEDs are addressed																		
LED Function	00h = LED off Override																		
	01h - FAh = LED BLINKING override. The off duration is specified by the value of this byte and the on duration is specified by the value of byte Off Duration. Both values specify the time in tens of milliseconds (10 ms –2.5 s)																		
	FBh = Reserved																		
	FCh = LED state restored to Local Control state																		
	FDh - FEh = Reserved																		
	FFh = LED on Override																		
On Duration	LED on-time in tens of milliseconds if (1 ≤ LED Function Byte ≤ FAh) Otherwise, this field is ignored and is set to 0h.																		
Get LED state	0x07	\$ i2ctransfer -y \$I2C w2@0x2d 0x07 0x00 r6 LED ID = 0x00 Response: 0x01 0xff 0x00 0x00 0x00 0x6f LED Status = 0x01(override state) LED FUNCTION = 0xff (LED is on) LED ON Duration = 0x00 Override state LED FUNCTION = 0x00 Override Control on Duration = 0x00 PEC: 0x6f (0x5b 0x01 0xff 0x00 0x00 0x00)																	

		<table><tr><th>Byte</th><th>Data Definition</th></tr><tr><td rowspan="3">LED States</td><td>[7:2] - Reserved</td></tr><tr><td>[1] - 1b if override state has been enabled</td></tr><tr><td>[0] - 1b if CMM has a Local Control state</td></tr><tr><td rowspan="4">Local Control LED Function</td><td>00h = LED is off. This is the default value if the CMM does not support Local Control, like the Yellow LED.</td></tr><tr><td>01h - FAh = LED is BLINKING. The off duration is specified by the value of this bytes, and the on duration is secified by the value of "Local Control On Duration". Both values specify the time in tens of milliseconds(10ms - 2.5s)</td></tr><tr><td>FBh - FEh = Reserved</td></tr><tr><td>FFh = LED is on</td></tr><tr><td>Local Control On Duration</td><td>The LED on-time in tens of milliseconds if (01h ≤ Local Control LED Function Byte ≤ FAh). This byte is ignored and is set to 0h if the Local Control LED Function byte is not a value in range 01h - FAh</td></tr><tr><td rowspan="4">Override state LED Function</td><td>00h = LED override state is off.</td></tr><tr><td>01h - FAh = LED override state is BLINKING. The off duration is specified by the value of this byte and the on duration is specified by the value of "Override State On Duration". Both values specify the time in tens of milliseconds (10 ms–2.5 s).</td></tr><tr><td>FBh - FEh = Reserved</td></tr><tr><td>FFh = LED override state is on</td></tr><tr><td>Override Control On Duration</td><td>The LED on-time in tens of milliseconds if (01h ≤ Override Control LED Function Byte ≤ FAh). This byte is ignored and is set to 0h if the Override Control LED Function byte is not a value in range 01h - FAh</td></tr></table>	Byte	Data Definition	LED States	[7:2] - Reserved	[1] - 1b if override state has been enabled	[0] - 1b if CMM has a Local Control state	Local Control LED Function	00h = LED is off. This is the default value if the CMM does not support Local Control, like the Yellow LED.	01h - FAh = LED is BLINKING. The off duration is specified by the value of this bytes, and the on duration is secified by the value of "Local Control On Duration". Both values specify the time in tens of milliseconds(10ms - 2.5s)	FBh - FEh = Reserved	FFh = LED is on	Local Control On Duration	The LED on-time in tens of milliseconds if (01h ≤ Local Control LED Function Byte ≤ FAh). This byte is ignored and is set to 0h if the Local Control LED Function byte is not a value in range 01h - FAh	Override state LED Function	00h = LED override state is off.	01h - FAh = LED override state is BLINKING. The off duration is specified by the value of this byte and the on duration is specified by the value of "Override State On Duration". Both values specify the time in tens of milliseconds (10 ms–2.5 s).	FBh - FEh = Reserved	FFh = LED override state is on	Override Control On Duration	The LED on-time in tens of milliseconds if (01h ≤ Override Control LED Function Byte ≤ FAh). This byte is ignored and is set to 0h if the Override Control LED Function byte is not a value in range 01h - FAh
Byte	Data Definition																					
LED States	[7:2] - Reserved																					
	[1] - 1b if override state has been enabled																					
	[0] - 1b if CMM has a Local Control state																					
Local Control LED Function	00h = LED is off. This is the default value if the CMM does not support Local Control, like the Yellow LED.																					
	01h - FAh = LED is BLINKING. The off duration is specified by the value of this bytes, and the on duration is secified by the value of "Local Control On Duration". Both values specify the time in tens of milliseconds(10ms - 2.5s)																					
	FBh - FEh = Reserved																					
	FFh = LED is on																					
Local Control On Duration	The LED on-time in tens of milliseconds if (01h ≤ Local Control LED Function Byte ≤ FAh). This byte is ignored and is set to 0h if the Local Control LED Function byte is not a value in range 01h - FAh																					
Override state LED Function	00h = LED override state is off.																					
	01h - FAh = LED override state is BLINKING. The off duration is specified by the value of this byte and the on duration is specified by the value of "Override State On Duration". Both values specify the time in tens of milliseconds (10 ms–2.5 s).																					
	FBh - FEh = Reserved																					
	FFh = LED override state is on																					
Override Control On Duration	The LED on-time in tens of milliseconds if (01h ≤ Override Control LED Function Byte ≤ FAh). This byte is ignored and is set to 0h if the Override Control LED Function byte is not a value in range 01h - FAh																					
Set temperature going high threshold	0x08	\$ i2ctransfer -y \$I2C w8@0x2d 0x08 0xff 0x97 0xff 0x36 0x00 0x00 0xd1 Inlet Temperature = 0xff97 / 10(signed -105/10) = -10.5 Outlet Temperature = 0xff36 / 10(signed -202/10) = -20.2 Reserved Heatsink Temperature = 0x00000 PEC: 0xd1(0x5a 0x08 0xff 0x97 0xff 0x36 0x00 0x00)																				
Get Temperature going high threshold	0x09	\$ i2ctransfer -y \$I2C w1@0x2d 0x09 r7 Response: 0xff 0x97 0xff 0x36 0xff 0xff 0x50 Inlet Temperature = 0xff97 / 10(signed -105/10) = -10.5 Outlet Temperature = 0xff36 / 10(signed -202/10) = -20.2 Reserved Heatsink Temperature = 0xffff(N/A) PEC: 0x50(0x5b 0xff 0x97 0xff 0x36 0xff 0xff)																				
Set fan speed going low threshold	0x0a	\$ i2ctransfer -y \$I2C w5@0x2d 0x0a 0x3c 0x1e 0xc8 0x57 M = 0x3c (DEC 60) Local Fan Speed Threshold = M * 0x1e = 1800rpm Manual Fan Speed Threshold = M * 0xc8 = 12000rpm PEC: 0x57(0x5a 0x0a 0x3c 0x1e 0xc8)																				
Get fan speed going low threshold	0x0b	\$ i2ctransfer -y \$I2C w1@0x2d 0x0b r4 Response: 0x3c 0x1e 0xc8 0x9a M = 0x3c (DEC 60) Local Fan Speed Threshold = M * 0xc8 = 1800rpm Manual Fan Speed Threshold = M * 0xc8 = 12000rpm PEC: 0x9a (0x5b 0x3c 0x1e 0xc8)																				
Set the power off delay time	0x0c	\$ i2ctransfer -y \$I2C w3@0x2d 0x0c 0x08 0x67 Power Off Delay Timer = 0x08(8 minutes)																				

		PEC: 0x67 (0x5a 0x0c 0x08)
Get the power off delay timer	0x0d	\$ i2cttransfer -y \$I2C w1@0x2d 0x0d r2 Response: 0x08 0xa3 Power Off Delay Timer = 0x08(8 minutes) PEC: 0xa3(0x5b 0x08)
Set the battery voltage stable stage timer	0x0e	\$ i2cttransfer -y \$I2C w3@0x2d 0x0e 0x28 0xad Battery Voltage Stable Stage Timer = 0x28(40 seconds) PEC: 0xad (0x5a 0x0e 0x28)
Get the battery voltage stable stage timer	0x0f	\$ i2cttransfer -y \$I2C w1@0x2d 0x0f r2 Response: 0x28 0x43 Battery Voltage Stable Stage Timer = 0x28(40 seconds) PEC: 0x43(0x5b 0x28)
Set the power on quarantine stage timer	0x10	\$ i2cttransfer -y \$I2C w3@0x2d 0x10 0xf0 0x2a Pwr on Quarantine Stage Timer = 0xf0(240 second) PEC: 0x2a (0x5a 0x10 0xf0)
Get the power one quarantine stage timer	0x11	\$ i2cttransfer -y \$I2C w1@0x2d 0x11 r2 Response: 0xf0 0x45 Pwr on Quarantine Stage Timer = 0xf0(240 second) PEC: 0x45(0x5b 0xf0)
Restore to factory default settings	0x12	\$ i2cttransfer -y \$I2C w2@0x2d 0x12 0xf0 This command will restore the factory default settings, including the power off delay timer, the default fan speed going low threshold, the default inlet/outlet temperature going high threshold, etc. PEC: 0xf0(0x5a 0x12)
Query the manufacturer	0x13	\$ i2cttransfer -y \$I2C w1@0x2d 0x13 r7 Response: 0x41 0x44 0x4c 0x49 0x4e 0x4b 0x88 ID = 0x41 0x44 0x4c 0x49 0x4e 0x4b (ASCII "ADLINK") PEC: 0xb6 (0x5b 0x41 0x44 0x4c 0x49 0x4e 0x4b)
Query the firmware version	0x14	\$ i2cttransfer -y \$I2C w1@0x2d 0x14 r11 Response: 0x39 0x30 0x36 0x30 0x39 0x34 0x39 0x37 0x35 0x36 0x7a There convert to ASCII code string: "9060949756" PEC: 0x7a (0x5b 0x39 0x30 0x36 0x30 0x39 0x34 0x39 0x37 0x35 0x36)
Set the Fan Level	0x15	\$ i2cttransfer -y \$I2C w3@0x2d 0x15 0x1e 0xef

		Set fan level to 0x1e(DEC 30) duty cycle PEC: 0xef (0x5a 0x1e 0xef)
Get the Fan Level	0x16	\$ i2cttransfer -y \$I2C w1@0x2d 0x16 r3 Response: 0xff 0x1e 0x45 Override fan level= 0xff(Local control, fan curve mode) Local control fan level(duty) = 0x1e(duty cycle value in fan curve mode) PEC: 0x45 (0x5b 0xff 0x1e)
Set the default Fan curve	0x17	\$ i2cttransfer -y \$I2C w3@0x2d 0x17 0x0a 0xa9 Set default fan curve to 0x0a(DEC 10) PEC: 0xa9 (0x5a 0x17 0x0a)
Get the default Fan curve	0x18	\$ i2cttransfer -y \$I2C w1@0x2d 0x18 r2 Response: 0x0a 0xad Default fan curve = 0x0a(DEC 10) PEC: 0xad (0x5b 0x0a)
Get the serial number	0x20	\$ i2cttransfer -y \$I2C w1@0x2d 0x20 r13 Response: 0x30 0x31 0x32 0x33 0x34 0x35 0x36 0x37 0x38 0x39 0x41 0x42 0x1e There convert to ASCII code string: “0123456789AB” PEC: 0x1e (0x5b 0x30 0x31 0x32 0x33 0x34 0x35 0x36 0x37 0x38 0x39 0x41 0x42)

8 Appendix – References

NA

9 External References

NA

End of document